Projet Programmation Orientée Objet

Gestion agence de location immobilière

présenté par SIMSEK Ilayda et Chantal FURNON

Sommaire

Sommaire **2**
Analyse et Conception **3**
Gestion des clients **4**
Créer un client 4
Modifier un client 5
Supprimer un client 6
Afficher clients par ordre alphabétique 7
Gestion des biens **8**
Créer un bien 8
Modifier un bien (dans la class Fenetre) 9
Supprimer un bien (dans la class Fenetre) 9
Gestion des locations **11**
Ajouter une location (dans la class Fenetre) 11
Méthode 1: Dans l'onglet "Location" 11
Méthode 2: Le bouton 13
Afficher les biens loués avec leur date début d'une location et biens non loués (dans class Fenetre) 13
Supprimer une location 14
Faire une recherche pour trouver un bien par rapport à sa surface totale 15
Avoir un menu qui propose les fonctionnalités précédentes 17
Interface graphique 18
Gestion des exceptions **21**
Dans le main 21
Dans la class Fenetre 21
Dans la class Bien 26
Conclusion **27**

SIMSEK Ilayda
Chantal FURNON

Analyse et Conception

Avant de lancer l'environnement de développement (IDE), nous avons consacré une phase essentielle à la réflexion et à la modélisation de notre structure. Ce brouillon préparatoire a été le socle de notre travail en binôme.

Ce que ce travail de conception nous a permis de réaliser :

- **Identification de la structure :** Nous avons listé tous les attributs (types long, double, boolean, String, int) pour identifier ce qui appartient à la classe mère (Bien) et ce qui est spécifique aux classes filles (Appartement, Maison, Garage).
- **Clarification des tâches :** En visualisant toutes les fonctionnalités demandées (Création, Ajout, Modification, Suppression, Recherche, Affichage), nous avons pu lister précisément chaque méthode à coder.
- **Division du travail au sein du binôme :** Grâce à cette vue d'ensemble, nous avons pu répartir les tâches de manière équitable :
    - ➢ Ilayda : class client, class Agence
    - ➢ Chantal: class Bien (mère), class (filles) garage, appartement, maison

Une fois que toutes nos class étaient prêtes, on les a mises ensemble. On a ensuite travaillé à deux sur le code pour l'améliorer, parfois pour ajouter des éléments selon les consignes, parfois pour corriger le code/déboguer. Ce document de travail nous a servi de fil rouge tout au long du projet pour vérifier que chaque condition d'achèvement était bien respectée et que nos codes respectifs pouvaient s'assembler sans erreur.

SIMSEK Ilayda
Chantal FURNON

Gestion des clients

Créer un client

class Agence

```
public void ajouterClient(Client c) {
    listeClients.add(c);
}
```

class Fenetre

```
private void ajouterClient() {
    JTextField champNom = new JTextField();
    JTextField champPrenom = new JTextField();
    JTextField champMail = new JTextField();
    JTextField champTelephone = new JTextField();

    Object[] message = {
        "Nom:", champNom,
        "Prénom:", champPrenom,
        "Email:", champMail,
        "Téléphone:", champTelephone
    };

    int option = JOptionPane.showConfirmDialog(this, message, "Ajouter un client", JOptionPane.OK_CANCEL_OPTION);

    if (option == JOptionPane.OK_OPTION) {
        try {
            String nom = champNom.getText().trim();
            String prenom = champPrenom.getText().trim();
            String mail = champMail.getText().trim();
            String telephone = champTelephone.getText().trim();

            if (nom.isEmpty() || prenom.isEmpty() || mail.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Tous les champs sont obligatoires !", "Erreur", JOptionPane.ERROR_MESSAGE);
                return;
            }

            Client nouveauClient = new Client(nom, prenom, mail, telephone);
            agence.ajouterClient(nouveauClient);
            rafraichirListes();


            JOptionPane.showMessageDialog(this, "Client ajouté avec succès !", "Succès", JOptionPane.INFORMATION_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Le numéro de téléphone doit être un nombre !", "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

On a utilisé des boîtes de dialogue (JOptionPane) pour que ce soit plus visuel et on a mis un message d'erreur pour que tous les champs soient bien remplis.
L'ajout de la méthode *rafraichirListes()* : nous permet d'afficher le nouveau client directement.

Le test : *if (option == JOptionPane.OK_OPTION)*, permet d'éviter les erreurs de manipulation et facilite l'utilisateur dans ses démarches.
L'utilisation de String (pour téléphone) au lieu de int ou bien long comme il se doit en informatique est pour assurer un bon affichage du numéro de téléphone.
La méthode try…catch (Exception ex), permet la gestion d'erreurs, notamment pour téléphone si il y a la saisie de lettres et non de chiffres un message d'erreurs s'affiche.
L'utilisation de *.trim()* permet d'enlever les espaces inutiles mis par erreur par l'utilisateur.
Lorsque l'on ajoute un nouveau client, la commande add permet de la mettre dans la liste des clients et *rafraichirListes()* permet l'affichage actualisé dans le bon ordre.

SIMSEK Ilayda
Chantal FURNON

Modifier un client

class Agence

```
public boolean modifierClient (String nomClient, String nouveauNom, String nouveauPrenom, String nouveauEmail, String nouveauTel) {
    Client c = LireClient (nomClient);
    if (c != null) {
        c.setNom(nouveauNom);
        c.setPrenom(nouveauPrenom);
        c.setEmail(nouveauEmail);
        c.setTelephone(nouveauTel);
        System.out.println("Client modifié avec succès !");
        return true;
    } else {
        System.out.println("Erreur : Client introuvable !");
        return false;
    }
}
```

class Fenetre

```
private void modifierClient() {
    String selection = listeClients.getSelectedValue();

    if (selection == null) {
        JOptionPane.showMessageDialog(this, "Veuillez sélectionner un client dans la liste.", "Aucune sélection", JOptionPane.WARNING_MESSAGE);
        return;
    }

    String nomClient = selection.split(" ")[0];

    Client clientATrouver = agence.LireClient(nomClient);
    if (clientATrouver != null) {
        JTextField champNom = new JTextField(clientATrouver.getNom());
        JTextField champPrenom= new JTextField(clientATrouver.getPrenom());
        JTextField champMail = new JTextField(clientATrouver.getEmail());
        JTextField champTelephone = new JTextField(clientATrouver.getTelephone());


    Object[] message = {
        "Nouveau Nom:", champNom,
        "Nouveau Prenom:", champPrenom,
        "Nouveau Email:", champMail,
        "Nouveau Téléphone:", champTelephone
    };

    int option = JOptionPane.showConfirmDialog(this, message, "Modifier le client " + nomClient, JOptionPane.OK_CANCEL_OPTION);

    if (option == JOptionPane.OK_OPTION) {
            clientATrouver.setNom(champNom.getText().trim());
            clientATrouver.setPrenom(champPrenom.getText().trim());
            clientATrouver.setEmail(champMail.getText().trim());
            clientATrouver.setTelephone(champTelephone.getText().trim());
            rafraichirListes();

            JOptionPane.showMessageDialog(this, "Client modifié avec succès !", "Succès", JOptionPane.INFORMATION_MESSAGE);
        }
    } else {
            JOptionPane.showMessageDialog(this, "Le numéro de téléphone doit être un nombre !", "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
```

Lorsque l'on veut modifier un client, l'utilisateur doit simplement sélectionner le client dans la liste et le code récupère toutes ses informations (Nom, Prénom, Email, Téléphone) pour les afficher dans des cases. Il reste plus qu'à modifier l'élément que l'on veut, cela évite de tout recommencer à 0. Cette modification se fait avec la méthode "set". Encore une fois, nous avons rajouté *rafraichirListes()* pour une visualisation plus simple et rapide des changements.
La ligne de code *selection.split(" ")[0]* : elle permet de retrouver le client en coupant la ligne sélectionnée pour récupérer seulement le nom de famille.
JTextField champNom = new JTextField(clientATrouver.getNom()); : grâce à cette méthode les cases à remplir ne sont pas vides mais elles récupèrent les données pour modifier celle que nous voulons seulement.

Pour la class Agence, l'utilisation de la méthode boolean permet de renvoyer true ou false, si il renvoie true alors on peut modifier les cases mais si elle renvoie false alors il envoie un message d'erreur ("Erreur: Client introuvable !")

Supprimer un client

class Fenetre

```
private void supprimerClient() {
    String selection = listeClients.getSelectedValue();

    if (selection == null) {
        JOptionPane.showMessageDialog(this, "Veuillez sélectionner un client dans la liste.", "Aucune sélection", JOptionPane.WARNING_MESSAGE);
        return;
    }

    String nomClient = selection.split(" ")[0];

    int confirmation = JOptionPane.showConfirmDialog(this,
        "Êtes-vous sûr de vouloir supprimer le client " + nomClient + " ?",
        "Confirmation",
        JOptionPane.YES_NO_OPTION);

    if (confirmation == JOptionPane.YES_OPTION) {
        Client clientASupprimer = null;
        for (Client c : agence.getListeClients()) {
            if (c.getNom().equalsIgnoreCase(nomClient)) {
                clientASupprimer = c;
                break;
            }
        }

        if (clientASupprimer != null) {
            agence.getListeClients().remove(clientASupprimer);
            rafraichirListes();
            JOptionPane.showMessageDialog(this, "Client supprimé avec succès !", "Succès", JOptionPane.INFORMATION_MESSAGE);
        }
    }
}
```

class Agence

```
public boolean supprimerClient (String nomClient) {
    Client c = LireClient (nomClient);
    if (c != null) {
        listeClients.remove(c);
        System.out.println("Client supprimé avec succès !");
        return true;
    } else {
        System.out.println("Erreur: Client introuvable !");
        return false;
    }
}
```

On a mis en place un système de confirmation pour éviter les erreurs d'inattention.
La condition `if` (confirmation == JOptionPane.YES_OPTION) permet de lancer la suppression seulement si l'utilisateur appuie sur Oui.
Pour la suppression, il faut d'abord sélectionner un client avant de pouvoir le supprimer avec la méthode remove.
Si la suppression se réalise alors un message de succès s'affiche.
rafraichirListes() permet de mettre à jour la nouvelle liste de clients.
La méthode remove dans la class Agence retire l'objet de la list ArrayList.

Afficher clients par ordre alphabétique

class Agence

```
public void trierClients() {
    Collections.sort(listeClients, new Comparator<Client>() {
        @Override
        public int compare(Client c1, Client c2) {
            return c1.getNom().compareToIgnoreCase(c2.getNom());
        }
    });
}
```

Pour un affichage des clients par ordre alphabétique, nous avons créé une méthode qui compare les noms deux par deux. L'utilisation de *compareToIgnoreCase* permet de faire un tri correct et structuré des noms.

SIMSEK Ilayda
Chantal FURNON

Gestion des biens

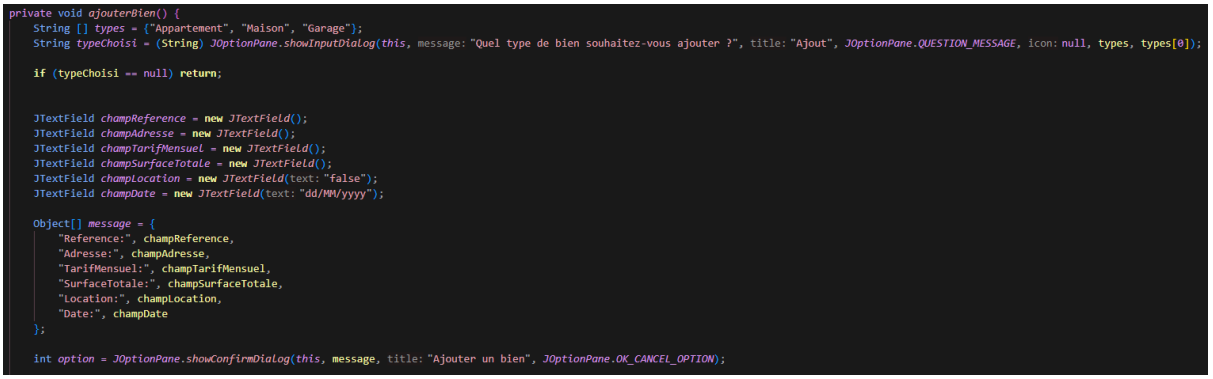
Créer un bien

Class Agence

```
public void ajouterBien(Bien b) {
    listeBiens.add(b);
}
```

Tout d'abord dans la classe agence, nous avons lié la méthode *ajouterBien()* à la liste bien avec l'argument *.add* pour s'assurer que le bien qu'on ajoute soit bien enregistré et placé dans la liste adéquate.

Class Fenetre



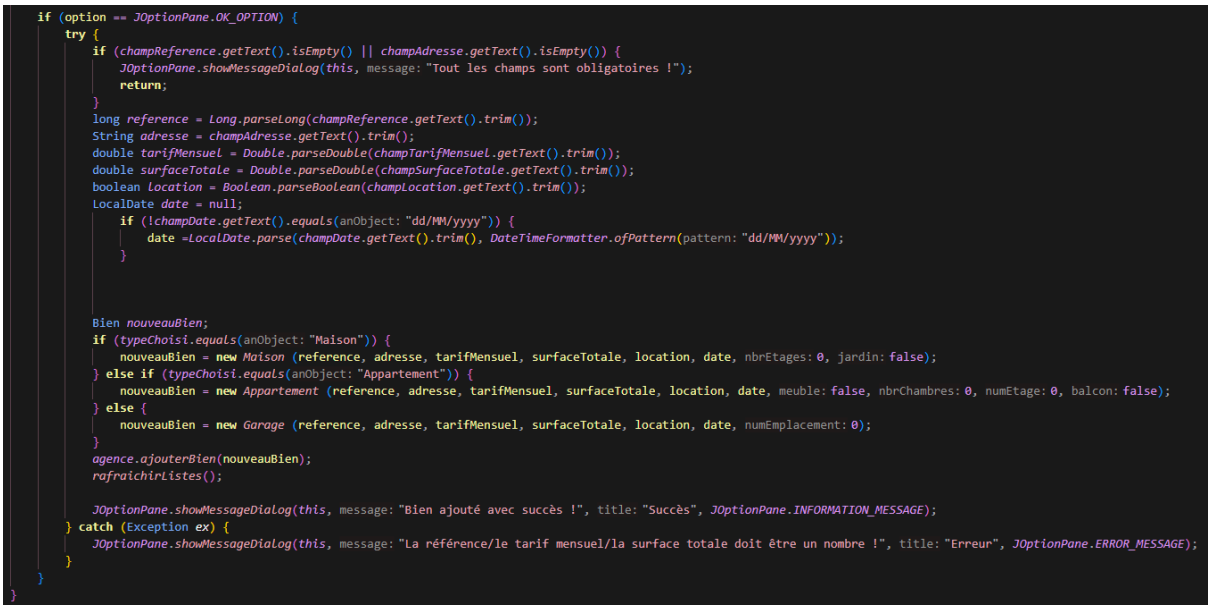
```
private void ajouterBien() {
    String [] types = {"Appartement", "Maison", "Garage"};
    String typeChoisi = (String) JOptionPane.showInputDialog(this, message: "Quel type de bien souhaitez-vous ajouter ?", title: "Ajout", JOptionPane.QUESTION_MESSAGE, icon: null, types, types[0]);

    if (typeChoisi == null) return;


    JTextField champReference = new JTextField();
    JTextField champAdresse = new JTextField();
    JTextField champTarifMensuel = new JTextField();
    JTextField champSurfaceTotale = new JTextField();
    JTextField champLocation = new JTextField(text: "false");
    JTextField champDate = new JTextField(text: "dd/MM/yyyy");

    Object[] message = {
        "Reference:", champReference,
        "Adresse:", champAdresse,
        "TarifMensuel:", champTarifMensuel,
        "SurfaceTotale:", champSurfaceTotale,
        "Location:", champLocation,
        "Date:", champDate
    };

    int option = JOptionPane.showConfirmDialog(this, message, title: "Ajouter un bien", JOptionPane.OK_CANCEL_OPTION);
```



```
    if (option == JOptionPane.OK_OPTION) {
        try {
            if (champReference.getText().isEmpty() || champAdresse.getText().isEmpty()) {
                JOptionPane.showMessageDialog(this, message: "Tout les champs sont obligatoires !");
                return;
            }
            long reference = Long.parseLong(champReference.getText().trim());
            String adresse = champAdresse.getText().trim();
            double tarifMensuel = Double.parseDouble(champTarifMensuel.getText().trim());
            double surfaceTotale = Double.parseDouble(champSurfaceTotale.getText().trim());
            boolean location = Boolean.parseBoolean(champLocation.getText().trim());
            LocalDate date = null;
                if (!champDate.getText().equals(anObject: "dd/MM/yyyy")) {
                    date =LocalDate.parse(champDate.getText().trim(), DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy"));
                }



            Bien nouveauBien;
            if (typeChoisi.equals(anObject: "Maison")) {
                nouveauBien = new Maison (reference, adresse, tarifMensuel, surfaceTotale, location, date, nbrEtages: 0, jardin: false);
            } else if (typeChoisi.equals(anObject: "Appartement")) {
                nouveauBien = new Appartement (reference, adresse, tarifMensuel, surfaceTotale, location, date, meuble: false, nbrChambres: 0, numEtage: 0, balcon: false);
            } else {
                nouveauBien = new Garage (reference, adresse, tarifMensuel, surfaceTotale, location, date, numEmplacement: 0);
            }
            agence.ajouterBien(nouveauBien);
            rafraichirListes();

            JOptionPane.showMessageDialog(this, message: "Bien ajouté avec succès !", title: "Succès", JOptionPane.INFORMATION_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, message: "La référence/le tarif mensuel/la surface totale doit être un nombre !", title: "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Ensuite, nous nous occupons de l'aspect technique et esthétique dans la class Fenetre pour permettre à l'utilisateur d'ajouter toutes les informations nécessaires.

Modifier un bien (dans la class Fenetre)

```
private void modifierBien() {
    String selection = listeBiens.getSelectedValue();

    if (selection == null) {
        JOptionPane.showMessageDialog(this, message: "Veuillez sélectionner un bien dans la liste.", title: "Aucune sélection", JOptionPane.WARNING_MESSAGE);
        return;
    }

    long reference = Long.parseLong(selection.split(regex: ",")[0].replaceAll(regex: "[^0-9]", replacement: ""));


    Bien bienATrouver = agence.LireBien(reference);
    if (bienATrouver == null) return;

    JTextField champAdresse = new JTextField(bienATrouver.getAdresse());
    JTextField champTarifMensuel = new JTextField(String.valueOf(bienATrouver.getTarifMensuel()));
    JTextField champSurfaceTotale = new JTextField(String.valueOf(bienATrouver.getSurfaceTotale()));
    JTextField champLocation = new JTextField(String.valueOf(bienATrouver.isLocation()));
    JTextField champDate = new JTextField(bienATrouver.getDate() != null ? bienATrouver.getDate().format(DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy")) : "dd/MM/yyyy");

    Object[] message = {
        "Nouvelle adresse:", champAdresse,
        "Nouveau Tarif mensuel:", champTarifMensuel,
        "Nouveau Surface totale:", champSurfaceTotale,
        "Loacation ?:", champLocation,
        "Début date location:", champDate
    };

    int option = JOptionPane.showConfirmDialog(this, message, "Modifier le bien n° " + reference, JOptionPane.OK_CANCEL_OPTION);
```

```
    if (option == JOptionPane.OK_OPTION) {
        try {
            bienATrouver.setAdresse(champAdresse.getText().trim());
            bienATrouver.setTarifMensuel(Double.parseDouble(champTarifMensuel.getText().trim()));
            bienATrouver.setSurfaceTotale(Double.parseDouble(champSurfaceTotale.getText().trim()));
            bienATrouver.setLocation(Boolean.parseBoolean(champLocation.getText().trim()));
            if (!champDate.getText().equals(anObject: "dd/MM/yyyy")) {
                bienATrouver.setDate(LocalDate.parse(champDate.getText().trim(), DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy")));
            }
            rafraichirListes();

            JOptionPane.showMessageDialog(this, message: "Bien modifié avec succès !", title: "Succès", JOptionPane.INFORMATION_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, message: "La référence/le tarif mensuel/la surface totale doit être un nombre !", title: "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Pour cette méthode, nous traitons directement les données dans la class Fenetre.
Nous commençons par nous assurer que le bien à modifier est bien sélectionné en se basant sur la référence de celui-ci.
Une fois la référence récupérée par le logiciel, je fais en sorte d'utiliser le mot clé *new* pour modifier chacun des champs.

Supprimer un bien (dans la class Fenetre)

```
private void supprimerBien() {
    String selection = listeBiens.getSelectedValue();

    if (selection == null) {
        JOptionPane.showMessageDialog(this, message: "Veuillez sélectionner un bien dans la liste.", title: "Aucune sélection", JOptionPane.WARNING_MESSAGE);
        return;
    }

    String refStr = selection.split(regex: ":")[1].split(regex: " ")[1];

    int confirmation = JOptionPane.showConfirmDialog(this,
        "Êtes-vous sûr de vouloir supprimer le bien n°" + refStr + " ?",
        title: "Confirmation",
        JOptionPane.YES_NO_OPTION);

    if (confirmation == JOptionPane.YES_OPTION) {
        try {
            long reference = Long.parseLong(refStr);
            Bien bienASupprimer = null;

            for (Bien b : agence.getListeBiens()) {
                if (b.getReference() == reference) {
                    bienASupprimer = b;
                    break;
                }
            }

            if (bienASupprimer != null) {
                agence.getListeBiens().remove(bienASupprimer);
                rafraichirListes();
                JOptionPane.showMessageDialog(this, message: "Bien supprimé avec succès !", title: "Succès", JOptionPane.INFORMATION_MESSAGE);
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, message: "Erreur lors de la suppression.", title: "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

message de confirmation afin de permettre à l'utilisateur de revenir sur sa décision. Le bien sera supprimé de la liste associée *agence.listeBiens()*, on rafraîchit la liste à l'aide de la méthode *rafraichirListes()* et un message de confirmation du succès de la suppression s'affiche ou bien d'erreur de suppression en cas d'exception.

Gestion des locations

Ajouter une location (dans la class Fenetre)

Méthode 1: Dans l'onglet "Location"

```
private void ajouterLocation() {

    JTextField champReference = new JTextField();
    JTextField champNomClient = new JTextField();
    JTextField champDate = new JTextField(
        LocalDate.now().format(DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy"))
    );

    Object[] message = {
        "Référence du bien :", champReference,
        "Nom du client :", champNomClient,
        "Date (dd/MM/yyyy) :", champDate
    };

    int option = JOptionPane.showConfirmDialog(
        this,
        message,
        title: "Ajouter une location",
        JOptionPane.OK_CANCEL_OPTION
    );

    if (option == JOptionPane.OK_OPTION) {
        try {
            long reference = Long.parseLong(champReference.getText().trim());
            String nomClient = champNomClient.getText().trim();
            String date = champDate.getText().trim();

            if (nomClient.isEmpty()) {
                JOptionPane.showMessageDialog(this,
                    message: "Le nom du client est obligatoire.",
                    title: "Erreur",
                    JOptionPane.ERROR_MESSAGE);
                return;
            }

            Bien bien = agence.LireBien(reference);
            if (bien == null) {
                JOptionPane.showMessageDialog(this,
                    message: "Bien introuvable.",
                    title: "Erreur",
                    JOptionPane.ERROR_MESSAGE);
                return;
            }
```

```
            if (bien.isLocation()) {
                JOptionPane.showMessageDialog(this,
                    message: "Ce bien est déjà loué.",
                    title: "Erreur",
                    JOptionPane.WARNING_MESSAGE);
                return;
            }

            boolean succes = agence.louerBien(reference, nomClient, date);

            if (succes) {
                rafraichirListes();
                JOptionPane.showMessageDialog(this,
                    message: "Location ajoutée avec succès !",
                    title: "Succès",
                    JOptionPane.INFORMATION_MESSAGE);
            } else {
                JOptionPane.showMessageDialog(this,
                    message: "Client introuvable ou date invalide.",
                    title: "Erreur",
                    JOptionPane.ERROR_MESSAGE);
            }

        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this,
                message: "La référence doit être un nombre.",
                title: "Erreur",
                JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Nous affichons ici une petite fenêtre où la référence et le nom du client doivent être remplis, ainsi que le choix de la date du début de location (par défaut la date du jour).
Une fenêtre s'affiche pour signifier à l'utilisateur le succès de l'ajout ou bien une erreur
Cette méthode est accessible depuis l'onglet "Location".

Méthode 2: Le bouton

```
private void procederLocation() {
    String selection = listeBiens.getSelectedValue();
    if (selection == null) {
        JOptionPane.showMessageDialog(this, message: "Sélectionnez un bien dans la liste !");
        return;
    }

    try {
        String refStr = selection.split(regex: ":")[1].trim().split(regex: " ")[0];
        long ref = Long.parseLong(refStr);

        Bien b = agence.LireBien(ref);
        if (b != null && b.isLocation()) {
            JOptionPane.showMessageDialog(this,  "Ce bien est déjà loué par " + b.getLocataire().getNom(), title: "Bien indisponible", JOptionPane.WARNING_MESSAGE);
            return;
        }

        JTextField champNomClient = new JTextField();
        JTextField champDate = new JTextField(LocalDate.now().format(DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy")));

        Object[] message = {
            "Nom du client :", champNomClient,
            "Date (dd/MM/yyyy) :", champDate
        };

        int option = JOptionPane.showConfirmDialog(this, message, "Louer le bien n°" + ref, JOptionPane.OK_CANCEL_OPTION);

        if (option == JOptionPane.OK_OPTION) {
            boolean succes = agence.louerBien(ref, champNomClient.getText().trim(), champDate.getText().trim());
            if (succes) {
                rafraichirListes();
                JOptionPane.showMessageDialog(this, message: "Location enregistrée avec succès !");
            } else {
                JOptionPane.showMessageDialog(this, message: "Erreur : Client introuvable ou mauvais format de date.", title: "Erreur", JOptionPane.ERROR_MESSAGE);
            }
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, message: "Erreur lors de la récupération de la référence.");
    }
}
```

Ici, le système de sélection a été utilisé afin de n'avoir à remplir que le nom du client et le choix de la date. Une petite fenêtre s'ouvrira pour confirmer l'ajout de la location ou pour signaler une erreur.

Les deux méthodes ont été conservées afin d'avoir un menu organisé et intuitif.

Afficher les biens loués avec leur date début d'une location et biens non loués (dans class Fenetre)

```
private void afficherFiltreBiens(boolean seulementLoues) {
    modelBiens.clear();
    for (Bien b : agence.getListeBiens()) {
        if (b.isLocation() == seulementLoues) {
            modelBiens.addElement(b.toString());
        }
    }

    if (modelBiens.isEmpty()) {
        JOptionPane.showMessageDialog(this, message: "Aucun bien ne correspond à ce critère.");
        rafraichirListes();
    }
}
```

Cette méthode utilise la boucle *for* où l'on reprend la liste des biens par le getter *agence.getListeBiens()* et le booléen *b.isLocation().*

```java
@Override
public void actionPerformed(ActionEvent e) {
```

```java
    if (e.getSource().equals(itemBiensLoues)) {
        afficherFiltreBiens(seulementLoues: true);
    }

    if (e.getSource().equals(itemBiensNonLoues)) {
        afficherFiltreBiens(seulementLoues: false);
    }
```

Pour filtrer correctement, nous allons préciser dans la méthode *actionPerformed(ActionEvent e)* que le booléen soit true si l'on veut afficher que les biens loués, soit false si l'on veut afficher que les biens non loués.

Supprimer une location

```java
private void supprimerLocation() {
    String selection = listeBiens.getSelectedValue();

    if (selection == null) {
        JOptionPane.showMessageDialog(this, message: "Veuillez sélectionner un bien dans la liste.", title: "Aucune sélection", JOptionPane.WARNING_MESSAGE);
        return;
    }

    try {
        String refStr = selection.split(regex: ":")[1].split(regex: " ")[1];
        long reference = Long.parseLong(refStr);

        Bien bienTrouve = agence.LireBien(reference);

        if (bienTrouve == null) {
            JOptionPane.showMessageDialog(this, message: "Bien introuvable !", title: "Erreur", JOptionPane.ERROR_MESSAGE);
            return;
        }

        if (!bienTrouve.isLocation()) {
            JOptionPane.showMessageDialog(this,
                message: "Ce bien n'est pas actuellement loué.",
                title: "Pas de location",
                JOptionPane.INFORMATION_MESSAGE);
            return;
        }

        String nomLocataire = bienTrouve.getLocataire() != null ? bienTrouve.getLocataire().getNom() : "Inconnu";
        String dateDebut = bienTrouve.getDate() != null ? bienTrouve.getDate().format(DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy")) : "Inconnue";

        int confirmation = JOptionPane.showConfirmDialog(this,
            "Supprimer la location du bien n°" + reference + " ?\n" +
            "Locataire : " + nomLocataire + "\n" +
            "Depuis le : " + dateDebut,
            title: "Confirmation",
            JOptionPane.YES_NO_OPTION);
```

```java
        if (confirmation == JOptionPane.YES_OPTION) {
            bienTrouve.setLocation(location: false);
            bienTrouve.setLocataire(locataire: null);
            bienTrouve.setDate(date: null);

            rafraichirListes();
            JOptionPane.showMessageDialog(this,
                message: "Location supprimée avec succès !\nLe bien est maintenant disponible.",
                title: "Succès",
                JOptionPane.INFORMATION_MESSAGE);
        }

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this,
            message: "Erreur lors de la récupération de la référence.",
            title: "Erreur",
            JOptionPane.ERROR_MESSAGE);
    }
}
```

Nous réutilisons le système de sélection et allons vérifier à partir de la référence si le bien est loué. Si tel est le cas, le booléen *.setLocation* sera changé en *false*, et le locataire ainsi que la date en "*null*".
Une fenêtre de confirmation apparaîtra avant le processus afin de permettre à l'utilisateur d'annuler s'il le souhaite.

Faire une recherche pour trouver un bien par rapport à sa surface totale

```
private void rechercherBien() {
    String recherche = champRechercheBien.getText().trim();

    if (recherche.isEmpty()) {
        rafraichirListes();
        return;
    }

    try {
        double surfaceEntree = Double.parseDouble(recherche);
        modelBiens.clear();
        boolean trouve = false;

        for (Bien b : agence.getListeBiens()) {
            double surfaceBien = b.getSurfaceTotale();

            if (surfaceBien >= surfaceEntree && surfaceBien < surfaceEntree + 1) {
                modelBiens.addElement(b.toString());
                trouve = true;
            }
        }

        if (!trouve) {
            JOptionPane.showMessageDialog(this,
                message: "Aucun bien trouvé pour cette surface.",
                title: "Recherche",
                JOptionPane.INFORMATION_MESSAGE);
            rafraichirListes();
        }
```

```
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this,
                message: "Veuillez entrer uniquement un nombre (ex : 50).",
                title: "Erreur",
                JOptionPane.ERROR_MESSAGE);
        }
    }
```

Nous allons utiliser la liste *listeBiens()* ainsi que le getter *b.getSurfaceTotale()*. On utilise une boucle if afin de faire une comparaison entre la valeur de la surface totale du bien que l'on recherche et la valeur correspondant aux biens. Cette comparaison a été faite de sorte à ce

que la recherche soit souple. C’est à dire que l’on n’est pas obligé de mettre la valeur en entier avec les chiffres après la virgule, l’entier suffit.

Avoir un menu qui propose les fonctionnalités précédentes

```
private JMenuBar construireMenu() {
    JMenuBar bar = new JMenuBar();
    JMenu menuClient = new JMenu(s: "Client");
    JMenu menuBien = new JMenu(s: "Bien");
    JMenu menuLocation = new JMenu(s: "Location");

    bar.add(menuClient);
    bar.add(menuBien);
    bar.add(menuLocation);

    itemAjouterClient = new JMenuItem(text: "Ajouter");
    itemAjouterClient.addActionListener(this);
    itemModifierClient = new JMenuItem(text: "Modifier");
    itemModifierClient.addActionListener(this);
    itemSupprimerClient = new JMenuItem(text: "Supprimer");
    itemSupprimerClient.addActionListener(this);

    menuClient.add(itemAjouterClient);
    menuClient.add(itemModifierClient);
    menuClient.add(itemSupprimerClient);

    itemAjouterBien = new JMenuItem(text: "Ajouter");
    itemAjouterBien.addActionListener(this);
    itemModifierBien = new JMenuItem(text: "Modifier");
    itemModifierBien.addActionListener(this);
    itemSupprimerBien = new JMenuItem(text: "Supprimer");
    itemSupprimerBien.addActionListener(this);
    itemBiensLoues = new JMenuItem (text: "Afficher biens loués");
    itemBiensLoues.addActionListener(this);
    itemBiensNonLoues = new JMenuItem (text: "Afficher biens non loués");
    itemBiensNonLoues.addActionListener(this);

    menuBien.add(itemAjouterBien);
    menuBien.add(itemModifierBien);
    menuBien.add(itemSupprimerBien);
    menuBien.add(itemBiensLoues);
    menuBien.add(itemBiensNonLoues);

    itemAjouterLocation = new JMenuItem(text: "Ajouter");
    itemAjouterLocation.addActionListener(this);
    itemSupprimerLocation = new JMenuItem(text: "Supprimer");
    itemSupprimerLocation.addActionListener(this);

    menuLocation.add(itemAjouterLocation);
    menuLocation.add(itemSupprimerLocation);
```

```
    return bar;
}
```

Dans notre menu, nous avons 3 onglets : Client, Bien, et Location

Pour Client, nous avons mis les items Ajouter (*itemAjouterClient*), Modifier (*itemModifierClient*), et Supprimer (*itemSupprimerClient)*.

De même pour l'onglet Bien (*itemAjouterBien, itemModifierBien, itemSupprimerBien)*. Celui-ci a également les items pour afficher les biens loués (*itemBiensLoues*), et les biens non loués (*itemBiensNonLoues*).

Pour Location, il y a les items Ajouter (*itemAjouterLocation*), et Supprimer (*itemSupprimerLocation*).

Interface graphique

Pour établir l'interface graphique, nous avons fait deux panels avec un GridLayout.

Le panel gauche est dédié à la partie client, composé de :

- *labelRechercher* : pour afficher le label recherche
- *labelNomClient* : pour afficher le label du nom du client
- *champRechercheClient* : pour rentrer le nom/prénom du client que l'on recherche
- *btnRechercherClient* : un bouton pour faire la recherche du client
- *scrollPanelClients* : pour scroll dans le panel client
- *btnResetClients* : un bouton pour afficher en entier la liste de client
- *btnExporter* : un bouton pour créer un fichier csv de la liste de clients

Le panel droit est dédié à la partie bien, composé de :

- *labelRechercheBien* : pour afficher le label recherche
- *labelRefBien* : pour afficher le label de la surface totale du bien
- *champRechercheBien* : pour rentrer la surface totale du bien que l'on recherche
- *btnRechercheBien* : pour rechercher la surface totale du bien que l'on recherche
- *scrollPaneBien* : pour scroller dans la liste de bien à l'horizontale pour voir toutes les informations de la liste
- *btnResetBiens* : bouton pour afficher toute la liste en entier, indépendamment des filtres
- *btnLouer* : bouton pour louer le bien sélectionné

```java
private JPanel construirePanel() {
    JPanel panel = new JPanel(new GridLayout(rows: 1, cols: 2));
    JPanel panelGauche = new JPanel();
    panelGauche.setLayout(new BoxLayout(panelGauche, BoxLayout.PAGE_AXIS));
    JPanel panelDroit = new JPanel();
    panelDroit.setLayout(new BoxLayout(panelDroit, BoxLayout.PAGE_AXIS));
    panelDroit.setBorder(BorderFactory.createLineBorder(Color.BLACK, thickness: 2));
    panelGauche.setBorder(BorderFactory.createLineBorder(Color.BLACK, thickness: 2));
    panel.add(panelGauche);
    panel.add(panelDroit);

    JLabel labelRechercher = new JLabel(text: "RECHERCHER UN CLIENT");
    labelRechercher.setFont(new Font(name: "Arial", Font.BOLD, size: 14));
    labelRechercher.setAlignmentX(Component.CENTER_ALIGNMENT);

    JLabel labelNomClient = new JLabel(text: "Nom du client :");
    labelNomClient.setAlignmentX(Component.CENTER_ALIGNMENT);

    champRechercheClient.setMaximumSize(new Dimension(width: 250, height: 30));
    champRechercheClient.setAlignmentX(Component.CENTER_ALIGNMENT);

    JButton btnRechercherClient = new JButton(text: "Rechercher");
    btnRechercherClient.setAlignmentX(Component.CENTER_ALIGNMENT);
    btnRechercherClient.addActionListener(e -> rechercherClient());

    JButton btnLouer = new JButton(text: "Louer le bien sélectionné");
    btnLouer.setAlignmentX(Component.CENTER_ALIGNMENT);
    btnLouer.setBackground(new Color(r: 173, g: 216, b: 230));
    btnLouer.addActionListener(e -> procederLocation());


    JButton btnResetClients = new JButton (text: "Tout afficher");
    btnResetClients.setAlignmentX(Component.CENTER_ALIGNMENT);
    btnResetClients.addActionListener(e-> rafraichirListes());

    JButton btnResetBiens = new JButton (text: "Tout afficher");
    btnResetBiens.setAlignmentX(Component.CENTER_ALIGNMENT);
    btnResetBiens.addActionListener(e-> rafraichirListes());
```

```java
    JButton btnExporter = new JButton (text: "Exporter Clients (CSV)");
    btnExporter.setAlignmentX(Component.CENTER_ALIGNMENT);
    btnExporter.setBackground(new Color(r: 200, g: 230, b: 201));
    btnExporter.addActionListener(e-> {
        agence.creationFichierClient();
        JOptionPane.showMessageDialog(this, message: "Fichier CSV crée !");
    });

    for (Client c : agence.getListeClients()) {
        modelClients.addElement(c.toString());
    }

    JScrollPane scrollPaneClients = new JScrollPane(listeClients);
    scrollPaneClients.setPreferredSize(new Dimension(width: 250, height: 300));
    scrollPaneClients.setMaximumSize(new Dimension(width: 350, height: 300));
    scrollPaneClients.setAlignmentX(Component.CENTER_ALIGNMENT);

    panelGauche.add(Box.createRigidArea(new Dimension(width: 0, height: 10)));
    panelGauche.add(labelRechercher);
    panelGauche.add(Box.createRigidArea(new Dimension(width: 0, height: 15)));
    panelGauche.add(labelNomClient);
    panelGauche.add(Box.createRigidArea(new Dimension(width: 0, height: 5)));
    panelGauche.add(champRechercheClient);
    panelGauche.add(Box.createRigidArea(new Dimension(width: 0, height: 10)));
    panelGauche.add(btnRechercherClient);
    panelGauche.add(Box.createRigidArea(new Dimension(width: 0, height: 15)));
    panelGauche.add(scrollPaneClients);
    panelGauche.add(Box.createRigidArea(new Dimension(width: 0, height: 10)));
    panelGauche.add(btnResetClients);
    panelGauche.add(Box.createRigidArea(new Dimension(width: 0, height: 5)));
    panelGauche.add(btnExporter);

    JLabel labelRechercheBien = new JLabel(text: "RECHERCHER UN BIEN");
    labelRechercheBien.setFont(new Font(name: "Arial", Font.BOLD, size: 14));
    labelRechercheBien.setAlignmentX(Component.CENTER_ALIGNMENT);

    JLabel labelRefBien = new JLabel(text: "Surface totale du bien :");
    labelRefBien.setAlignmentX(Component.CENTER_ALIGNMENT);

    champRechercheBien.setMaximumSize(new Dimension(width: 250, height: 30));
    champRechercheBien.setAlignmentX(Component.CENTER_ALIGNMENT);

    JButton btnRechercherBien = new JButton(text: "Rechercher");
    btnRechercherBien.setAlignmentX(Component.CENTER_ALIGNMENT);
    btnRechercherBien.addActionListener(e -> rechercherBien());
```

```
    for (Bien b : agence.getListeBiens()) {
        modelBiens.addElement(b.toString());
    }

    JScrollPane scrollPaneBiens = new JScrollPane(listeBiens);
    scrollPaneBiens.setPreferredSize(new Dimension(width: 250, height: 300));
    scrollPaneBiens.setMaximumSize(new Dimension(width: 350, height: 300));
    scrollPaneBiens.setAlignmentX(Component.CENTER_ALIGNMENT);

    panelDroit.add(Box.createRigidArea(new Dimension(width: 0, height: 10)));
    panelDroit.add(labelRechercheBien);
    panelDroit.add(Box.createRigidArea(new Dimension(width: 0, height: 15)));
    panelDroit.add(labelRefBien);
    panelDroit.add(Box.createRigidArea(new Dimension(width: 0, height: 5)));
    panelDroit.add(champRechercheBien);
    panelDroit.add(Box.createRigidArea(new Dimension(width: 0, height: 10)));
    panelDroit.add(btnRechercherBien);
    panelDroit.add(Box.createRigidArea(new Dimension(width: 0, height: 15)));
    panelDroit.add(scrollPaneBiens);
    panelDroit.add(Box.createRigidArea(new Dimension(width: 0, height: 10)));
    panelDroit.add(btnResetBiens);
    panelDroit.add(Box.createRigidArea(new Dimension(width: 0, height: 5)));
    panelDroit.add(btnLouer);

    return panel;
}
```

Gestion des exceptions

Dans le main

```java
public void creationFichierClient() {
    this.trierClients();
    File f = new File(pathname: "fichierClient.csv");

    try (PrintWriter writer = new PrintWriter(f)) {
        writer.println(x: "Nom; Prenom; Email; Telephone");
        for (Client c :listeClients) {
            writer.println(c.getNom() + "; " + c.getPrenom() +"; " + c.getEmail() + "; " + c.getTelephone());
        }
        System.out.println(x: "Fichier crée avec succès !");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Dans la méthode de création de clients, l'exception sert uniquement à éviter que le programme plante si la création ou l'écriture du fichier échoue.

```java
public boolean louerBien(long ref, String nomClient, String dateLocation) {
    Bien bienTrouve = LireBien(ref);
    Client clientTrouve = LireClient(nomClient);

    if (bienTrouve != null && bienTrouve.isLocation()) {
        System.out.println("Erreur: Le bien n°" + ref + " est déjà loué !");
        return false;
    }

    if (bienTrouve != null && clientTrouve != null) {
        bienTrouve.setLocation(location: true);
        bienTrouve.setLocataire(clientTrouve);
        try {
            bienTrouve.setDate(LocalDate.parse(dateLocation, DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy")));
            System.out.println("Succès: Le bien n°" + ref + " est loué à " + nomClient);
            return true;
        } catch (Exception e) {
            System.out.println(x: "Erreur: format date !");
            return false;
        }
    }
    System.out.println(x: "Erreur: Bien ou Client introuvable !");
    return false;
}
```

Dans celle-ci, l'exception permet de gérer une erreur de format de date lors de la saisie afin d'éviter un crash du programme.

Dans la class Fenetre

Dans la méthode *rechercherBien()*

```
    if (!trouve) {
        JOptionPane.showMessageDialog(this,
            message: "Aucun bien trouvé pour cette surface.",
            title: "Recherche",
            JOptionPane.INFORMATION_MESSAGE);
        rafraichirListes();
    }

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
            message: "Veuillez entrer uniquement un nombre (ex : 50).",
            title: "Erreur",
            JOptionPane.ERROR_MESSAGE);
    }
}
```

Ici, l'exception permet d'intercepter une saisie invalide lorsque l'utilisateur n'entre pas un nombre pour la surface, afin d'éviter un crash du programme.

Dans la méthode *ajouterClient()*

```
    if (option == JOptionPane.OK_OPTION) {
        try {
            String nom = champNom.getText().trim();
            String prenom = champPrenom.getText().trim();
            String mail = champMail.getText().trim();
            String telephone = champTelephone.getText().trim();

            if (nom.isEmpty() || prenom.isEmpty() || mail.isEmpty()) {
                JOptionPane.showMessageDialog(this, message: "Tous les champs sont obligatoires !", title: "Erreur", JOptionPane.ERROR_MESSAGE);
                return;
            }

            Client nouveauClient = new Client(nom, prenom, mail, telephone);
            agence.ajouterClient(nouveauClient);
            rafraichirListes();


            JOptionPane.showMessageDialog(this, message: "Client ajouté avec succès !", title: "Succès", JOptionPane.INFORMATION_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, message: "Le numéro de téléphone doit être un nombre !", title: "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

L'exception permet d'éviter un crash si le numéro de téléphone saisi est invalide lors de la création du client.

Dans la méthode *ajouterBien()*

```
        int option = JOptionPane.showConfirmDialog(this, message, "Ajouter un bien", JOptionPane.OK_CANCEL_OPTION);

        if (option == JOptionPane.OK_OPTION) {
            try {
                if (champReference.getText().isEmpty() || champAdresse.getText().isEmpty()) {
                    JOptionPane.showMessageDialog(this, "Tout les champs sont obligatoires !");
                    return;
                }
                long reference = Long.parseLong(champReference.getText().trim());
                String adresse = champAdresse.getText().trim();
                double tarifMensuel = Double.parseDouble(champTarifMensuel.getText().trim());
                double surfaceTotale = Double.parseDouble(champSurfaceTotale.getText().trim());
                boolean location = Boolean.parseBoolean(champLocation.getText().trim());
                LocalDate date = null;
                    if (!champDate.getText().equals("dd/MM/yyyy")) {
                        date =LocalDate.parse(champDate.getText().trim(), DateTimeFormatter.ofPattern("dd/MM/yyyy"));
                    }


                Bien nouveauBien;
                if (typeChoisi.equals("Maison")) {
                    nouveauBien = new Maison (reference, adresse, tarifMensuel, surfaceTotale, location, date, 0, false);
                } else if (typeChoisi.equals("Appartement")) {
                    nouveauBien = new Appartement (reference, adresse, tarifMensuel, surfaceTotale, location, date, false, 0, 0, false);
                } else {
                    nouveauBien = new Garage (reference, adresse, tarifMensuel, surfaceTotale, location, date, 0);
                }
                agence.ajouterBien(nouveauBien);
                rafraichirListes();

                JOptionPane.showMessageDialog(this, "Bien ajouté avec succès !", "Succès", JOptionPane.INFORMATION_MESSAGE);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "La référence/le tarif mensuel/la surface totale doit être un nombre !", "Erreur", JOptionPan
            }
        }
    }
```

On utilise une condition if pour vérifier si la référence ou l'adresse sont vides avec (*isEmpty()*). Si c'est le cas, on arrête le processus avec return pour ne pas enregistrer des objets incomplets dans la liste de la class Agence.
La méthode *Double.parseDouble()* transforme le texte tapé (le format texte) par l'utilisateur en un nombre (format numérique) que l'ordinateur peut stocker et utiliser.
Les blocs *try…catch* (Exception ex) … permettent de régler les soucis des exceptions. Le programme essaye d'abord d'exécuter le bloc try, si une erreur de saisie par exemple fait surface alors le programme gère l'exception sans arrêter le programme et passe au bloc catch. Cela permet un bon fonctionnement du programme.
Dans les blocs catch on a pu mettre des *JOptionPane.showMessageDialog* qui permet d'alerter l'utilisateur avec une fenêtre contextuelle qui met en avant l'erreur qui s'est produite.

LocalDate.parse : elle nous a permis d'utiliser un format spécifique pour la date (dd/MM/yyyy) pour qu'elles soient toutes enregistrées et affichées de la même manière.
Si le format de date n'est pas respecté, alors le bloc catch est exécuté.

Les tests comme if (bienSelectionne != null) ou if (*champ.getText().isEmpty()*) permettent d'éviter les pointeurs nuls, le programme va vérifier qu'un élément est bien sélectionné avant de lancer une action comme la modification ou bien la suppression. Ce sont des tests de sécurité.

Dans *modifierBien()*

```
    if (option == JOptionPane.OK_OPTION) {
        try {
            bienATrouver.setAdresse(champAdresse.getText().trim());
            bienATrouver.setTarifMensuel(Double.parseDouble(champTarifMensuel.getText().trim()));
            bienATrouver.setSurfaceTotale(Double.parseDouble(champSurfaceTotale.getText().trim()));
            bienATrouver.setLocation(Boolean.parseBoolean(champLocation.getText().trim()));
            if (!champDate.getText().equals(anObject: "dd/MM/yyyy")) {
                bienATrouver.setDate(LocalDate.parse(champDate.getText().trim(), DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy")));
            }
            rafraichirListes();

            JOptionPane.showMessageDialog(this, message: "Bien modifié avec succès !", title: "Succès", JOptionPane.INFORMATION_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, message: "La référence/le tarif mensuel/la surface totale doit être un nombre !", title: "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

L'exception permet de gérer les erreurs de saisie lors de la modification d'un bien afin d'éviter un crash de l'application.

Dans *supprimerBien()*

```
            if (bienASupprimer != null) {
                agence.getListeBiens().remove(bienASupprimer);
                rafraichirListes();
                JOptionPane.showMessageDialog(this, message: "Bien supprimé avec succès !", title: "Succès", JOptionPane.INFORMATION_MESSAGE);
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, message: "Erreur lors de la suppression.", title: "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
```

L'exception permet de gérer une erreur de conversion de la référence du bien afin d'éviter un crash de l'application.

Dans *procederLocation()*

```
    if (option == JOptionPane.OK_OPTION) {
        boolean succes = agence.louerBien(ref, champNomClient.getText().trim(), champDate.getText().trim());
        if (succes) {
            rafraichirListes();
            JOptionPane.showMessageDialog(this, message: "Location enregistrée avec succès !");
        } else {
            JOptionPane.showMessageDialog(this, message: "Erreur : Client introuvable ou mauvais format de date.", title: "Erreur", JOptionPane.ERROR_MESSAGE);
        }
    }
} catch (Exception ex) {
    JOptionPane.showMessageDialog(this, message: "Erreur lors de la récupération de la référence.");
}
```

L'exception permet de gérer les erreurs liées à la récupération et à la conversion de la référence du bien afin d'éviter un crash de l'application.

Dans *ajouterLocation()*

```
        if (succes) {
            rafraichirListes();
            JOptionPane.showMessageDialog(this,
                message: "Location ajoutée avec succès !",
                title: "Succès",
                JOptionPane.INFORMATION_MESSAGE);
        } else {
            JOptionPane.showMessageDialog(this,
                message: "Client introuvable ou date invalide.",
                title: "Erreur",
                JOptionPane.ERROR_MESSAGE);
        }

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
            message: "La référence doit être un nombre.",
            title: "Erreur",
            JOptionPane.ERROR_MESSAGE);
    }
}
```

L'exception permet de gérer une erreur de conversion de la référence du bien afin d'éviter un crash de l'application lors de l'ajout d'une location.

Dans *supprimerLocation()*

```
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this,
        message: "Erreur lors de la récupération de la référence.",
        title: "Erreur",
        JOptionPane.ERROR_MESSAGE);
}
```

La gestion d'exception ici sert à éviter que l'application crash si le format de la chaîne sélectionnée est inattendu et/ou que la référence extraite n'est pas un nombre valide

Dans la class Bien

```
public void choisirDate() {
    if (location) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.println(x: "Entrez une date (jj/MM/aaaa) : ");
            if (scanner.hasNextLine()) {
                String dateStr = scanner.nextLine();
                this.date = LocalDate.parse(dateStr, DateTimeFormatter.ofPattern(pattern: "dd/MM/yyyy"));
            }
        } catch (DateTimeParseException e) {
            System.out.println(x: "Erreur: Format incorrect !");
        }
    } else {
        System.out.println(x: "Ce bien n'est pas en location.");
    }
}
```

Cette gestion d'exception sert à protéger contre les erreurs de saisie de date par l'utilisateur, à éviter un crash si le format est incorrect, à afficher un message clair plutôt qu'une erreur technique, ainsi qu'à fermer automatiquement le scanner grâce au try-with-resources.

Conclusion

Difficultés rencontrées :

Nous avons initialement mis le type long pour les numéros de téléphone, mais par la suite nous avons vu que cela supprimait les 0 au début donc nous l'avons modifié pour utiliser des String qui nous permettent de garder l'information exacte de la saisie à l'affichage.

Une des difficultés a été de faire apparaître les changements comme un ajout ou une suppression immédiatement à l'écran. Nous avons utilisé la méthode *rafraichirListes()* qui vide la liste visuelle et la recharge avec les nouvelles données de l'agence pour que l'utilisateur voit le résultat sans attendre.

Une autre difficulté a été d'extraire la référence d'un bien ou le nom du client à partir du texte affiché dans la liste. Nous avons dû utiliser des manipulations de chaînes de caractères avec *split* et *replaceAll* pour isoler l'identifiant et pouvoir retrouver l'objet réel dans l'agence afin de pouvoir le modifier ou le supprimer correctement.

Ce que l'on a fait :

Utilisation des ArrayList : pour pouvoir ajouter autant de clients que nous souhaitons et pouvoir les supprimer sans difficultés.

Pour que les informations soient lisibles dans l'interface, nous avons redéfini la méthode *toString()* dans nos classes (Client, Maison, Appartement, Garage). Cela permet de choisir les informations que nous voulons afficher dans les listes.

Utilisation de DefaultListModel<String> : c'est ce qui rend l'interface vivante, quand nous modifions le DefaultListModel avec la méthode *rafraichirListes()* la fenêtre se met à jour automatiquement. De plus, l'utilisation de *.clear()* pour vider et *.addElement()* pour remplir permet l'affichage exact des ArrayList.

Nous avons personnalisé les boîtes de dialogue JOptionPane en y intégrant directement nos propres champs de saisie (JTextField). Cette approche permet de créer des formulaires d'ajout ou de modification rapides sans avoir à multiplier les fenêtres complexes.

Dans les méthodes *supprimerClient()* ou *modifierBien()*, nous utilisons la méthode *.split()*. Comme la JList n'affiche que du texte avec *toString()*, nous utilisons split pour découper la ligne sélectionnée et récupérer uniquement l'élément que l'on veut (comme le nom ou la référence).

Nous avons intégré une fonctionnalité d'exportation. Le programme transforme l'ArrayList de clients en un fichier texte structuré (CSV) utilisable dans un tableur Excel. Cela montre la capacité du logiciel à communiquer avec d'autres outils.

Pour la recherche, nous avons utilisé *.toLowerCase()* et *.contains()* pour pouvoir afficher tous les clients qui correspondent à la recherche écrite.

Possibles améliorations :
On pourrait faire en sorte d'améliorer l'expérience utilisateur et l'ergonomie du logiciel.

Aide supplémentaire :

- w3schools : https://www.w3schools.com/colors/default.asp
- oracle : DefaultTableModel (Java SE 21 & JDK 21)
- Eclipse : Help - Eclipse Platform